

CSCI-UA 102 — Quiz 7

Collision Resolution

Name: _____ NetID: _____

Say I implement `hashCode` for `String` as:

```
public int hashCode() { return length(); }
```

I have a **hash map with an array of size 20** using **linear probing** and compression $h(k) = k.\text{hashCode}() \bmod 20$.

(3 points) I perform: `put("one",1), put("two",2), put("three",3), put("four",4), put("five",5), put("six",6), put("seven",7), put("eight",8), put("nine",9), put("ten",10)`. Draw the hash map. (The first entry is filled in for you.)

[illegible]

(1 point) I remove all the odd numbers: `remove("one"), remove("three"), remove("five"), remove("seven"), remove("nine")`. Draw the hash map. Use “A” for AVAILABLE (deleted) slots. (Slot 3 is filled in for you.)

[illegible]

(1 point) I add them back: `put("one",-1)`, `put("three",-3)`, `put("five",-5)`, `put("seven",-7)`, `put("nine",-9)`. Draw the hash map.

[illegible]

Solution

Hash values: "one"=3, "two"=3, "three"=5, "four"=4, "five"=4, "six"=3, "seven"=5, "eight"=5, "nine"=4, "ten"=3.

Part 1: After insertions. All 10 entries cluster into slots 3–12.

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19
			one	two	three	four	five	six	seven	eight	nine	ten							
			1	2	3	4	5	6	7	8	9	10							

Trace: “one” → 3. “two” → 3→**4**. “three” → 5. “four” → 4→5→**6**. “five” → 4→ ⋯ →**7**. “six” → 3→ ⋯ →**8**. “seven” → 5→ ⋯ →**9**. “eight” → 5→ ⋯ →**10**. “nine” → 4→ ⋯ →**11**. “ten” → 3→ ⋯ →**12**.

Part 2: After removals. Odd-valued entries become AVAILABLE sentinels (marked A).

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19
			A	two	A	four	A	six	A	eight	A	ten							
				2		4		6		8		10							

Each removal searches from $h(k)$, skipping AVAILABLE slots, until the key is found. For example, remove(“five”): $h = 4 \rightarrow$ “two” $\rightarrow$ A $\rightarrow$ “four” $\rightarrow$ “five” found at 7 $\rightarrow$ mark A.

Part 3: After re-insertions. During put, if we encounter an AVAILABLE slot, we remember it as firstAvail but **keep probing** to confirm the key is truly absent (reaching an empty slot). Then we insert at firstAvail.

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19
			one	two	three	four	five	six	seven	eight	nine	ten							
			−1	2	−3	4	−5	6	−7	8	−9	10							

Each re-inserted key lands in its original slot because that slot was the first AVAILABLE encountered, and the key was not found before reaching the empty slot at index 13.

For example, put(“one”,−1): $h = 3$, slot 3 is A $\rightarrow$ firstAvail=3, probe through all remaining slots, reach empty at 13 $\rightarrow$ key absent $\rightarrow$ insert at slot 3.

Key takeaway: The final table looks identical to Part 1 in structure, but the odd-number values are now negative. The AVAILABLE sentinel mechanism ensures that re-inserted keys reclaim their original positions.